

Mphasis Optimization Formulation

Abhay Shankar K — CS21BTECH11001
Anirudh Srinivasan — CS20BTECH11059
Gautam Singh — CS21BTECH11018

6th December 2023

1 Rules

1.1 Rule Set for PNR Scoring (PNR Individual)

1. SSR (Special Service Request):

- **Name:** SSR
- **Condition:** $\text{PNR.SSR} = \text{SSR}$
- **Score:** 200

2. Cabin:

- **Name:** Cabin
- **Conditions:** $\text{PNR.Cabin} = \text{F, J, Y}$
- **Score:** 1500 - 2000 (Dynamic scoring)

1.2 Rule Set for Flight Scoring

1. Arrival Delay:

- **Name:** Arrival Delay
- **Conditions:** $\text{Flight.proposed Arrival Delay} \leq 6/12/24/48 \text{ hours}$
- **Score:** 70/50/40/30 (Sample scoring)

1.3 Quality Score Grading for Flight Selection

1.4 Rules for Alternate Flight Selection and Connections

1.5 Downgrade and Upgrade Rules

1.6 Cabin Class Mapping

1.7 Solution Ranking Factors

1.8 Journey-Based Solution Scoring Mechanism

2 DQM

This is not a strict specification. There may be some inconsistencies.

Naturally, the version in `final.ipynb` is the correct one.

We are given a vector of PNR's `PNR`, Flights `Flights`, and cancelled flights `Cancelled`.

2.1 Things we compute

- Airport-flight directed graph, $\mathcal{A}$. Vertices are airports, edges are flights. Represent as adjacency matrix.
 - Each edge has a weight, which is the score of the flight.
 - For representing different citypair travel, add edges with negative weights between every pair of nodes? This precludes us from using simpler routing algos.
 - Maximise weight instead of minimising. Might still need DP.

$$A_{ij} = \begin{cases} 1 & \exists f : f.\text{origin} = i \wedge f.\text{destination} = j \\ 0 & \text{otherwise} \end{cases}$$

- Flight-flight directed graph, $\mathcal{F}$. Vertices are flights, edges exist if passenger can get off one and onto another. Represent as adjacency matrix.

This need not be a preprocessing step, but would appear to be most convenient there.

$$F_{ij} = \begin{cases} 1 & \text{Flights}[j].\text{std} - \text{Flights}[i].\text{eta} \in [1, 12] \wedge \text{Flights}[j].\text{origin} = \text{Flights}[i].\text{destination} \\ 0 & \text{otherwise} \end{cases}$$

- PNR vs flight path map, $\mathcal{P}$. This may be represented as a matrix of PNR vs Flight, where the element is 1 if the flight is in the path for the PNR, and 0 otherwise.

$$P_{ij} = \begin{cases} 1 & \text{Flight}[j] \in \text{PNR}[i].\text{flightpath} \\ 0 & \text{otherwise} \end{cases}$$

Sequence number thing did not work.

Note: This is the solution. As such, none of the flights should be in the cancelled list.

- A list of impacted PNR indices, $\mathcal{P}'$.

$$\forall i \in \text{PNR}, i \in \mathcal{P}' \iff i.\text{flightpath} \cap \text{Cancelled} \neq \emptyset$$

- A flight-flight graph relating arrival times (refer code), D (D for delta).

2.2 Constraints

- Flight cancellation enforced:

$$\sum_{i,j} \mathbb{1}_{Flights[j] \in Cancelled} P_{ij} = 0$$

- Flight capacity constraint.

$$\sum_i P_{ij} PNR[i].pax \leq Flights[j].capacity$$

- Flight connection constraint: Each PNR's flightpath must be in the flight-flight graph.

– Discrete variable cases

- * Say maximum 5 segments allowed. Each P_{ij} has 5 variants $1.. = 5$ and the 0, and each must appear at most once.

$$\forall i \sum_j P_{ij} :: k \leq 1 \quad \forall k$$

- * Segment k cannot appear without segment k - 1.

$$\forall i \sum_j P_{ij} :: k \leq \sum_j P_{ij} :: k - 1 \quad \forall k \in 2..5$$

- * If segment k appears, then flight connection. For every i, x, y, if $P_{ix} :: k$ and $P_{iy} :: k + 1$ then $F_{xy} = 1$.

$$\forall i, k, \sum_{x,y} P_{ix} :: k \cdot P_{iy} :: (k + 1) \cdot (F_{xy} - 1) = 0$$

This cannot be expressed as a constraint, so it will be added to the objective function we seek to *maximize*, as a term

$$\forall i, k, \lambda_c \left[\sum_{x,y} P_{ix} :: k \cdot P_{iy} :: (k + 1) \cdot (F_{xy} - 1) \right]$$

We need to provide a high λ_c to reduce likelihood of violation.
Feature, not bug: Sometimes, this may get violated if score is very high. This is good.

- Flight delay constraint:

$$\forall i \in \mathcal{P}' \bigwedge_j D[Flights[j]][PNR[i].last] == 1$$

- XXED: When VPNR has XX'ed a flight, should not get reassigned to that flight.

2.3 Objective Function

$$E(\text{whatever}) = \sum_i (PNR[i].score \cdot T_0 - \lambda_1 T_1 + \lambda_2 T_2)$$

where

$$\begin{aligned} T_0 &= \sum_j P_{ij} (Flight[j].score - \lambda_0) \\ T_1 &= \sum_j P_{ij} (Flights[j].eta - PNR[i].sta) \\ T_2 &= \sum_j \mathbb{1}_C P_{ij} \\ C &= Flights[j].origin == PNR[i].origin \\ &\quad || Flights[j].destination == PNR[i].destination \end{aligned}$$

λ_0 penalizes number of flights *per* *PNR*. The value of λ_0 should be such that two flights combined can never have a higher score than a single flight.

3 Transplant

A Discrete Quadratic Model is an optimisation model where each decision variable may have one or more variants (aka cases) with an objective function of the form

$$Z = \sum_i a_i(x_i) + \sum_i \sum_j b_{ij}(x_i, x_j) + C$$

where

- x_i are the decision variables
- a_i and b_{ij} are functions that map between the variants of the decision variables.

We have used such a model to solve the given problem, employing several intermediate data structures for a clean formulation. These are

- $\mathcal{F}$: A directed graph where each node is a flight, and an edge $a \rightarrow b$ exists if the following conditions hold.
 - The destination of a must be the source of b
 - The departure time of b is within 1 to 12 hours after the arrival time of a .
- $\mathcal{D}$: Another directed flight graph. An edge $a \rightarrow b$ exists if The departure time of b is less than 72 hours after the arrival time of a .

- **PNR:** A list of virtual PNRs. Each PNR in the input is split into multiple contiguous sequences of flights, each of which corresponds into a VPNR.
- **flights:** A list of flights. Read from input.
- We also accept a list of cancelled flights from the user.

The output of the model is $\mathcal{P}$, a VPNR-vs-Flight matrix. $\mathcal{P}_{ij}$ is the sequence number of flight j in the journey of VPNR i , or 0 not applicable.